NutriTrackWeb

Technische Dokumentation

NutriTrackWeb – Webanwendung für Ernährungsberatung

Name	Stefany Castelão dos Santos
Klasse	3. IT INF – Labor
Gruppe	3APP1
Datum	08.11.2026
Lehrer	Herr Platzer
Projekt	NutriTrackWeb

Inhaltsverzeichnis

1 Überblick und Architektur 3
1.1 Bausteine 3
1.2 Ablauf einer Anfrage 3
1.3 Rollen 4
2 Glossar 4
3 Projektstruktur 5
3.1 Dateien und Ordner 5
3.2 CSS und JavaScript 6
4 Datenbank 6
4.1 Tabellen 7
4.2 Dritte Normalform 8
4.3 Status-Codes 8
5 Sicherheit und Eingabeprüfung 9
5.1 Maßnahmen im Überblick 9
5.2 Eingabeprüfung 10
5.3 Uploads und geschützte Ordner 10
5.4 Testdatensätze für XSS-Schutz 10
6 Installation und Start 11
6.1 Schritte 11
6.2 Einstellung in der php.ini 12
6.3 HTTPS lokal (mkcert) 12
Zertifikat 13
6.4 Demo-Konten 13
7 Grenzen und Erweiterungen 14
7.1 Bekannte Grenzen der Version 1 14
7.2 Spätere Erweiterungen 14

1 Überblick und Architektur

NutriTrackWeb ist eine PHP-Webanwendung, in der eine Ernährungsberaterin ihre Kunden verwaltet. Die PHP-Seiten sprechen über PDO mit einer MariaDB-Datenbank und liefern fertiges HTML an den Browser. Was die Fachbegriffe bedeuten, steht in Kapitel 2.

1.1 Bausteine

Baustein	Verwendet	Aufgabe
Programmiersprache	PHP, ohne Framework (eine Datei pro Seite)	Logik und Ausgabe der Seiten
Datenbank	MariaDB, Datenbank *mydb*	Speichert Benutzer, Kunden, Termine, Nachrichten, Zahlungen u. a.
Datenzugriff	PDO mit Prepared Statements	Sicherer Zugriff auf die Datenbank
Webserver	Apache (XAMPP, Windows) mit HTTPS	Liefert die Seiten aus
Oberfläche	HTML, CSS, etwas JavaScript	Darstellung; Farben und Logo sind in den Einstellungen änderbar
Sprachen	Englisch, Deutsch, Portugiesisch	Die Texte stehen als Array *$t* in jeder PHP-Datei

1.2 Ablauf einer Anfrage

1. Der Browser ruft eine PHP-Seite auf, z. B. *client_home.php*.
2. Die Seite lädt *config.php*: Dort werden die Session gestartet (30 Minuten Timeout) und die Datenbankverbindung *$pdo* aufgebaut.
3. Die Seite prüft, ob jemand eingeloggt ist und welche Rolle er hat (*users.role*: 0 = User, 1 = Admin). Ohne passende Rolle geht es zurück zum Login.
4. Bei Formularen (POST) wird zuerst das CSRF-Token geprüft, dann werden die Eingaben validiert.
5. Die Seite liest oder schreibt Daten mit Prepared Statements. Zusammengehörige Schreibvorgänge laufen in einer Transaktion (bei einem Fehler *rollBack*).
6. Nach dem Speichern leitet die Seite per Redirect weiter (PRG-Muster), damit ein Neuladen nichts doppelt speichert.
7. Alle ausgegebenen Texte gehen durch *e()* (= *htmlspecialchars*), damit kein fremder Code im Browser läuft.

1.3 Rollen

Rolle	Darf	Darf nicht
Admin (Ernährungsberaterin)	Registrierungen prüfen sowie Kunden, Termine, Dokumente, Mahlzeitenfotos, Nachrichten, Zahlungen und Einstellungen verwalten	–
User (Kunde)	Eigene Daten, Termine, Dokumente, Mahlzeitenfotos, Nachrichten und Zahlungen sehen und bearbeiten	Admin-Seiten öffnen; fremde Daten oder Dateien sehen

Kunden registrieren sich selbst. Der Zugang gilt erst nach der Freigabe durch den Admin (*clients.approval_status*: 0 Pending, 1 Approved, 2 Rejected, 3 MoreInfos).

2 Glossar

Die wichtigsten Begriffe, einfach erklärt:

Begriff	Einfach erklärt
Webserver (Apache)	Das Programm, das die Seiten ausliefert, wenn der Browser danach fragt. Wie ein Kellner, der die Bestellung entgegennimmt und das Essen bringt.
PHP	Eine Programmiersprache, die auf dem Server läuft. PHP baut die Seite zusammen, der Browser bekommt nur das fertige HTML.
XAMPP	Ein Paket für den eigenen Rechner mit Apache, MariaDB, PHP und phpMyAdmin in einer Installation.
MariaDB	Der Datenbank-Server: der „Aktenschrank“, in dem alle Daten in Tabellen gespeichert sind. Es ist eine freie Weiterentwicklung von MySQL; das „MySQL“ im XAMPP ist in Wirklichkeit MariaDB.
SQL	Die Sprache, mit der man Daten aus der Datenbank liest und schreibt (z. B. *SELECT*, *INSERT*, *UPDATE*).
PDO	PHP Data Objects: der Standardweg von PHP, mit einer Datenbank zu sprechen. Wie ein einheitlicher Stecker, der den sicheren Weg mit Prepared Statements bietet.
Prepared Statement	Ein SQL-Befehl mit Platzhaltern (*?*). Die Werte werden getrennt gesendet. Wie ein Formular mit Lücken: Was in die Lücke geschrieben wird, kann den Text des Formulars nicht ändern.

Begriff	Einfach erklärt
Session	Ein Merkzettel auf dem Server für jeden Besucher. Der Browser trägt nur eine Nummer (wie eine Garderobennummer), die Daten bleiben auf dem Server.
Hash	Eine Art Fingerabdruck eines Passworts. Man kann ihn berechnen, aber nicht zum Passwort zurückrechnen.
CSRF	Eine fremde Webseite versucht, in Ihrem Namen ein Formular an unsere Seite zu schicken. Schutz: ein geheimes Token im Formular.
XSS	Jemand schreibt Programmcode (z. B. <*script*>) in ein Textfeld, damit er bei anderen im Browser läuft. Schutz: Ausgaben werden als reiner Text angezeigt.
SQL-Injection	Jemand schreibt SQL-Befehle in ein Feld, damit die Datenbank sie ausführt. Schutz: Prepared Statements.
Transaktion	Eine Gruppe von Speichervorgängen, die nur gemeinsam gelten: alles oder nichts.
.htaccess	Eine kleine Konfigurationsdatei für Apache. Hier sperrt sie den direkten Zugriff auf vertrauliche Uploads und verhindert Programmcode in den Upload-Ordnern.
HTTPS und Zertifikat	HTTPS verschlüsselt die Verbindung zwischen Browser und Server. Das Zertifikat ist der „Ausweis“ des Servers, damit der Browser ihm vertraut.
mkcert	Ein Werkzeug, das ein Zertifikat für den eigenen Rechner erstellt, dem der Browser auf diesem Rechner vertraut.
phpMyAdmin	Eine Weboberfläche, um die Datenbank anzusehen, zu importieren und zu exportieren.

3 Projektstruktur

3.1 Dateien und Ordner

Das Projekt liegt in *C:\xampp\htdocs\nutritrackweb*. Alle 24 PHP-Dateien liegen im Hauptordner. Dazu kommen der Ordner *icons/* (SVG-Symbole) und der Ordner *uploads/* (hochgeladene Dateien).

Bereich	Dateien	Aufgabe
Grundlage	*config.php*	Session-Start, Timeout, Datenbankverbindung (PDO)
Zugang	*index.php*, *login.php*, *logout.php*, *register.php*	Startseite, Anmeldung, Abmeldung, Selbstregistrierung

Bereich	Dateien	Aufgabe
Admin	*admin_dashboard.php*, *registration_review.php*, *clients.php*, *appointments.php*, *documents.php*, *meal_photos.php*, *messages.php*, *payments.php*, *settings.php*	Übersicht, Registrierungen prüfen, Kunden, Termine, Dokumente, Mahlzeitenfotos, Nachrichten, Zahlungen, Einstellungen
User	*client_home.php*, *client_status.php*, *client_profile.php*, *client_appointments.php*, *client_documents.php*, *client_meal_photos.php*, *client_messages.php*, *client_payments.php*	Startseite, Freigabestatus, Profil, Termine, Dokumente, Mahlzeitenfotos, Nachrichten, Zahlungen
Dateizugriff	*download.php*, *view_image.php*	Einziger Weg zu hochgeladenen Dateien, mit Rechteprüfung
Ordner	*icons/*, *uploads/documents/*, *uploads/meal_photos/*, *uploads/profile/*	Symbole; geschützte Upload-Ordner

Jede Seite hat denselben Aufbau: oben die PHP-Logik (Rolle prüfen, POST verarbeiten, Daten laden), darunter das HTML. Die Texte für Englisch, Deutsch und Portugiesisch stehen am Anfang jeder Datei im Array *$t*. Der Quellcode ist durchgehend auf Deutsch kommentiert.

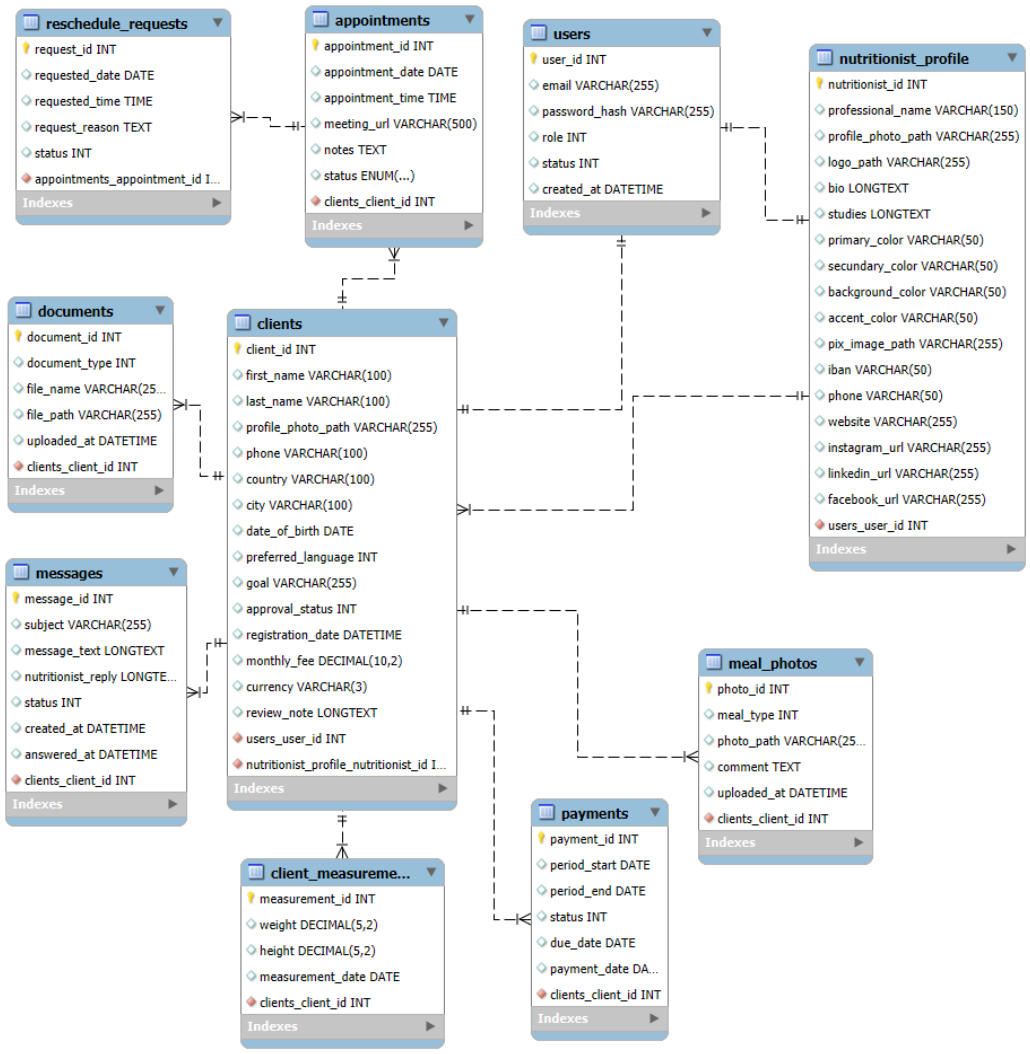
3.2 CSS und JavaScript

CSS und JavaScript stehen nicht in eigenen Dateien, sondern in jeder PHP-Seite selbst: das CSS in einem *<style>*-Block im Kopf der Datei, das JavaScript in einem *<script>*-Block am Ende. Beim CSS gibt es einen Grund dafür: Die Farben kommen aus der Datenbank (Tabelle *nutritionist_profile*). PHP schreibt sie beim Laden der Seite als CSS-Variablen (*--primary*, *--accent* usw.) in den Block. So ändert sich das Aussehen der ganzen Anwendung, sobald die Admin in den Einstellungen andere Farben wählt.

> **Hinweis:** Eigene Dateien (*style.css*, *app.js*) wären die professionellere Lösung, weil gleiche Regeln nur einmal stehen und der Browser die Dateien zwischenspeichern kann. Das ist für eine spätere Version geplant (siehe Kapitel 7).

4 Datenbank

Die Datenbank *mydb* hat 10 Tabellen (InnoDB, utf8) und erfüllt die dritte Normalform. Der komplette Export liegt in *nutritrackweb_database_dump.sql*.



ER-Diagramm der Datenbank mydb (MySQL Workbench)

4.1 Tabellen

Tabelle	Inhalt	Verbindung (Fremdschlüssel)
users	Login: E-Mail, *password_hash*, Rolle, Status	–
nutritionist_profile	Name, Bio, Farben, Logo, IBAN, Kontaktlinks der Beraterin	*users_user_id*
clients	Stammdaten, Ziel, Freigabestatus, Monatsbeitrag, Währung	*users_user_id*, *nutritionist_profile_nutritionist_id*
client_measurements	Gewicht, Größe, Datum	*clients_client_id*
appointments	Datum, Uhrzeit, Meeting-Link, Notizen, Status	*clients_client_id*

Tabelle	Inhalt	Verbindung (Fremdschlüssel)
reschedule_requests	Terminverschiebung: Wunschtermin, Grund, Status	*appointments_appointment_id*
documents	PDF-Dateien (Name, Pfad, Typ, Datum)	*clients_client_id*
meal_photos	Mahlzeitenfotos mit Kommentar	*clients_client_id*
messages	Nachricht des Kunden und Antwort der Beraterin	*clients_client_id*
payments	Zahlungszeitraum, Fälligkeit, Zahlungsdatum, Status	*clients_client_id*

4.2 Dritte Normalform

- Jede Tabelle hat genau einen Primärschlüssel aus einer Spalte (z. B. *client_id*).
- Jede Information steht nur einmal: Der Betrag einer Zahlung wird nicht doppelt gespeichert, sondern kommt aus *clients.monthly_fee*.
- Land und Stadt sind freie Textfelder und keine eigenen Tabellen, weil sie in der Anwendung nur angezeigt werden.
- Status-Werte sind Zahlencodes, die im PHP-Code erklärt werden (siehe 4.3).

Lesen und Schreiben laufen in allen Seiten über PDO. Schreibvorgänge über mehrere Tabellen (z. B. die Registrierung mit *users*, *clients* und Bluttest in *documents*) sind in einer Transaktion: Entweder wird alles gespeichert oder nichts.

4.3 Status-Codes

Feld	Werte
users.role	0 User, 1 Admin
clients.approval_status	0 Pending, 1 Approved, 2 Rejected, 3 MoreInfos
messages.status	0 Neu, 1 Beantwortet, 2 Geschlossen
payments.status	0 Open, 1 Paid, 2 Cancelled
reschedule_requests.status	0 Pending, 1 Approved, 2 Rejected
appointments.status	scheduled, rescheduled, completed, cancelled (Text)

5 Sicherheit und Eingabeprüfung

5.1 Maßnahmen im Überblick

Gefahr	Maßnahme	Einfach erklärt
Passwörter lesbar	*password_hash()* beim Speichern, *password_verify()* beim Login (*register.php*, *login.php*). Nur der Hash steht in *users.password_hash*. Mindestlänge 8 Zeichen.	Das Passwort wird nie gespeichert, nur ein „Fingerabdruck“. Beim Login wird der Fingerabdruck der Eingabe verglichen. Wird die Datenbank gestohlen, bleiben die Passwörter unlesbar.
Gestohlene Sitzung	*session_regenerate_id(true)* nach dem Login. Nach 30 Minuten ohne Aktivität *session_unset()* und *session_destroy()* (alle Seiten).	Nach dem Login gibt es eine neue „Garderobennummer“, eine alte Nummer ist wertlos. Nach 30 Minuten Pause wird die Nummer ungültig.
Gefälschte Formulare (CSRF)	Zufälliges Token (*random_bytes*) in Session und Formular, Vergleich mit *hash_equals()* (alle Seiten mit POST).	Jedes Formular enthält ein geheimes Token, das nur unsere Seite kennt. Passt es nicht, wird das Formular abgelehnt.
Fremder Code in der Ausgabe (XSS)	Jede Ausgabe über *e()* = *htmlspecialchars()* (alle Seiten).	Wer *<script>* in ein Feld schreibt, sieht es danach als normalen Text. Es läuft nicht als Programm.
SQL-Injection	Nur Prepared Statements mit *?*-Platzhaltern (alle Seiten).	Der Befehl wird zuerst als Vorlage geschickt, die Werte danach getrennt. Die Datenbank behandelt Eingaben nur als Daten, nie als Befehl.
Falsche Rolle, fremde Daten	Jede Seite prüft *users.role*. User sehen nur Zeilen mit der eigenen *client_id*.	Ein „Türsteher“ an jeder Seite: Ein User kommt nicht auf Admin-Seiten und nicht an die Daten anderer.
Halb gespeicherte Daten	Transaktion mit *rollBack* (*register.php*, *appointments.php*, *client_status.php*, *registration_review.php*).	Alles oder nichts: Bei einem Fehler wird alles zurückgenommen.
Doppeltes Absenden	Redirect nach dem Speichern (PRG-Muster, alle Formulare).	Nach dem Speichern lädt die Seite neu. F5 sendet das Formular nicht noch einmal.
Umgangene Browser-Prüfung	Prüfung zusätzlich im PHP, nicht nur mit *required* im Browser.	Die Browser-Prüfung kann man umgehen. Darum prüft auch der Server.

Gefahr	Maßnahme	Einfach erklärt
Abgehörte Verbindung	HTTPS mit Zertifikat (siehe 6.3).	Die Daten zwischen Browser und Server sind verschlüsselt.

5.2 Eingabeprüfung

Die Prüfungen laufen immer auf dem Server; im Browser gibt es sie nur zusätzlich (z. B. *required*). Beispiele aus *register.php*:

- Pflichtfelder dürfen nicht leer sein.
- E-Mail: *filter_var(..., FILTER_VALIDATE_EMAIL)* und höchstens 255 Zeichen.
- Passwort: mindestens 8 Zeichen.
- Ziel: höchstens 255 Zeichen.
- Datenschutzerklärung: Das Häkchen wird zusätzlich im PHP geprüft (*isset($_POST['privacy'])*).

Fehler erscheinen als Meldung in der gewählten Sprache. Die bereits eingegebenen Werte bleiben im Formular stehen.

5.3 Uploads und geschützte Ordner

- Die Ordner *uploads/documents* und *uploads/meal_photos* enthalten eine *.htaccess* mit *Require all denied*. Der Browser kann dort keine Datei direkt öffnen.
- Der Ordner *uploads/profile* (Profilfotos, Logo, PIX-QR-Code) hat eine *.htaccess*, die Programmdateien (z. B. PHP) sperrt und die Dateiliste ausblendet. Die Bilder selbst werden direkt angezeigt, weil sie nicht vertraulich sind (z. B. das Foto auf der Anmeldeseite).
- Dateien werden nur über *download.php* und *view_image.php* ausgeliefert. Beide prüfen Login und Berechtigung. Bei fremden oder fehlenden Dateien kommt dieselbe Antwort *404*, damit niemand erfährt, ob eine Datei existiert.
- Dateinamen werden zufällig vergeben. Ausgeliefert wird nur der *basename()* innerhalb des festen Ordners (*realpath*).
- Der Inhalt wird geprüft, nicht nur die Endung: ein PDF beginnt mit *%PDF-* (höchstens 10 MB), Bilder werden mit *getimagesize()* geprüft (höchstens 8 MB).

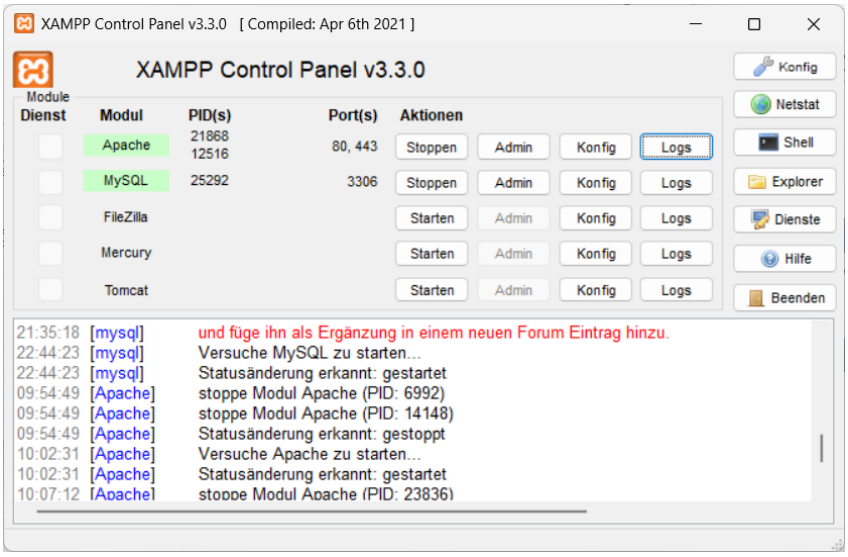
5.4 Testdatensätze für XSS-Schutz

In der Demo-Datenbank gibt es absichtlich einen Kunden mit dem Namen *Teste* und eine Nachricht mit *<script>alert(1)</script>*. Die Anwendung zeigt diesen Text unverändert an, ohne ihn auszuführen. Das ist der Nachweis, dass der Schutz vor XSS (*e()*) funktioniert.

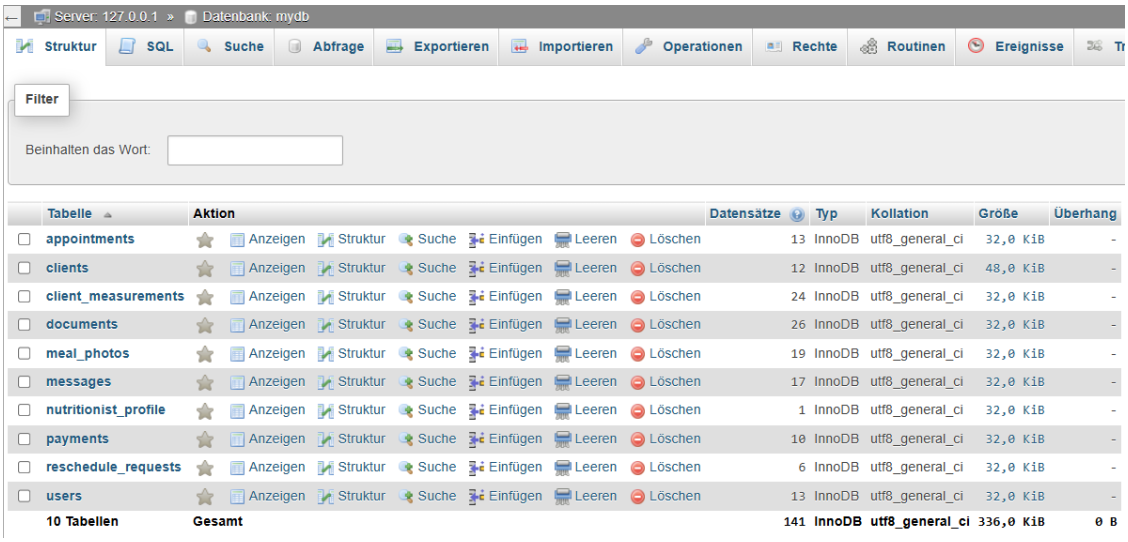
6 Installation und Start

6.1 Schritte

1. XAMPP installieren und im Control Panel **Apache** und **MySQL** (MariaDB) starten.
2. Den Projektordner nach *C:\xampp\htdocs\nutritrackweb* kopieren. Der Ordner *uploads/* muss die Unterordner *documents*, *meal_photos* und *profile* haben. In jedem liegt eine Datei *.htaccess*.
3. In phpMyAdmin (*http://localhost/phpmyadmin*) die Datenbank *mydb* anlegen und *nutritrackweb_database_dump.sql* importieren.
4. In *config.php* prüfen: Datenbank *mydb*, Benutzer *castelao*, leeres Passwort (nur für die lokale Entwicklung).
5. Im Browser *https://localhost/nutritrackweb/* öffnen und mit einem Demo-Konto anmelden (siehe 6.4).



XAMPP Control Panel mit laufendem Apache und MySQL



phpMyAdmin: Datenbank mydb mit ihren 10 Tabellen

6.2 Einstellung in der php.ini

upload_max_filesize und *post_max_size* müssen mindestens *10M* sein, sonst scheitert das Hochladen von PDFs. In dieser XAMPP-Installation stehen beide auf *40M*. Nach einer Änderung Apache neu starten.

6.3 HTTPS lokal (mkcert)

Damit *https://localhost* ohne Browser-Warnung funktioniert, wird mit dem Werkzeug mkcert ein lokal vertrauenswürdiges Zertifikat erzeugt.

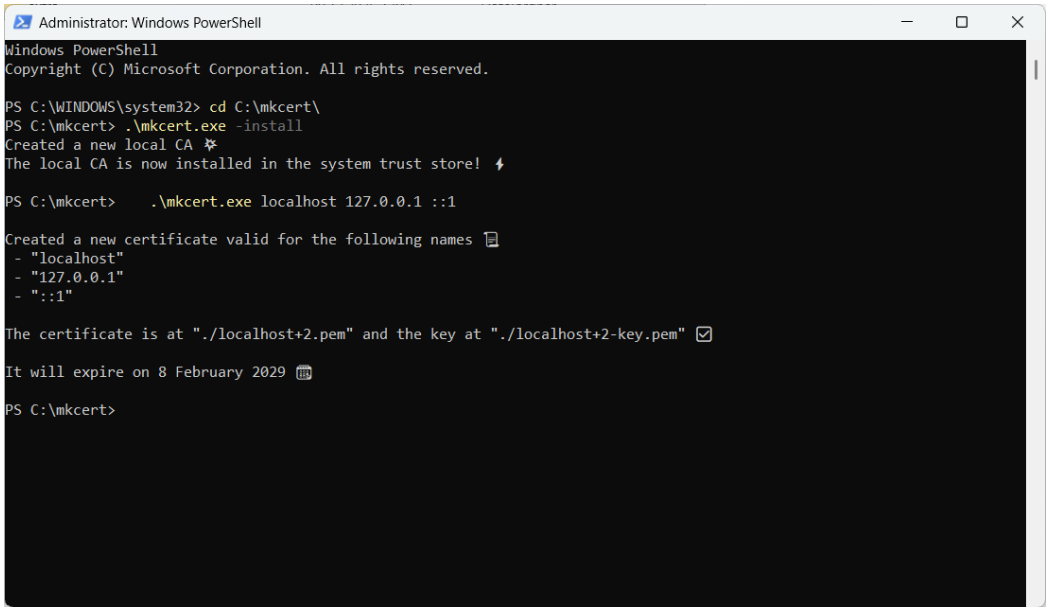
1. mkcert von der GitHub-Seite des Projekts herunterladen (Seite „Releases“, Datei für Windows, *mkcert-…-windows-amd64.exe*). Die Datei in den Ordner *C:\mkcert* legen und in *mkcert.exe* umbenennen.
2. **Windows PowerShell als Administrator** öffnen und die lokale Zertifizierungsstelle installieren:

```
cd C:\mkcert\
.\mkcert.exe -install
```

3. Das Zertifikat für *localhost* erzeugen:

```
.\mkcert.exe localhost 127.0.0.1 ::1
```

mkcert legt die Dateien *localhost+2.pem* (Zertifikat) und *localhost+2-key.pem* (Schlüssel) an.

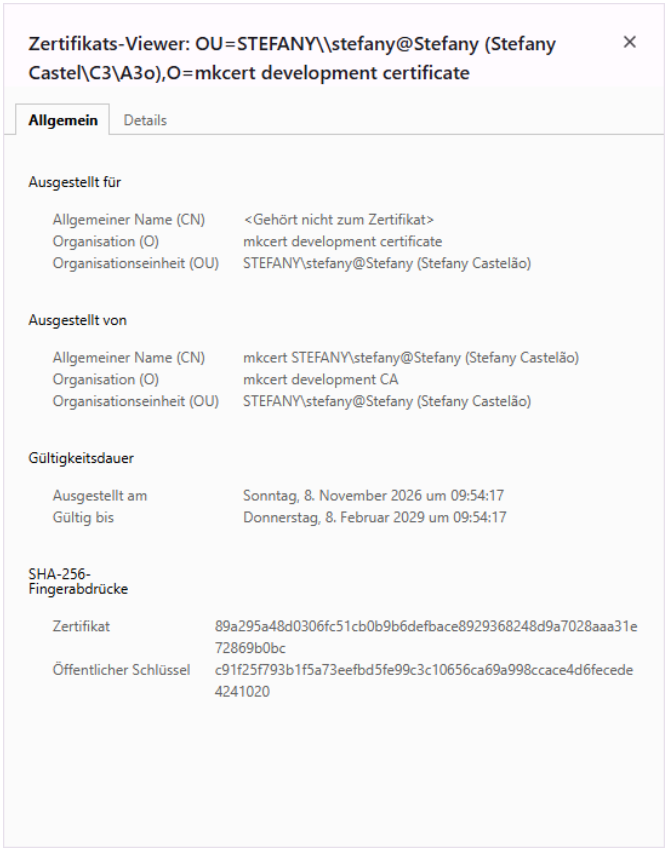


PowerShell: lokale Zertifizierungsstelle installieren und Zertifikat erzeugen

4. Beide Dateien in den XAMPP-Ordner von Apache kopieren und in *C:\xampp\apache\conf\extra\httpd-ssl.conf* bei *SSLCertificateFile* und *SSLCertificateKeyFile* eintragen.
5. Apache im XAMPP Control Panel neu starten und *https://localhost/nutritrackweb/* öffnen.

Zertifikat

Angabe	Wert
Ausgestellt von	mkcert development CA
Ausgestellt für	mkcert development certificate (Namen: *localhost*, *127.0.0.1*, *::1*)
Gültig ab	08.11.2026, 09:54 Uhr
Gültig bis	08.02.2029, 09:54 Uhr
Dateien	*localhost+2.pem*, *localhost+2-key.pem*



Zertifikats-Viewer im Browser mit Gültigkeitsdauer

Wichtig: Das Zertifikat ist nur auf dem Rechner vertrauenswürdig, auf dem *mkcert -install* ausgeführt wurde. Auf einem anderen Rechner zeigt der Browser eine Warnung. Die Präsentation erfolgt deshalb vom eigenen Notebook.

6.4 Demo-Konten

Der Datenbank-Export enthält Demo-Konten mit erfundenen Daten. Die Passwörter stehen in der Datei *Zugangsdaten.txt* im Abgabe-ZIP. Der Status eines Kontos (freigegeben, ausstehend, abgelehnt) kann sich beim Testen ändern.

Name	E-Mail	Rolle
Marianne Böhm, MSc	admin-demo@ntw.at	Admin
Lucas Pereira	user1-demo@ntw.at	User
Anna Fischer	user2-demo@ntw.at	User
Emily Carter	user3-demo@ntw.at	User
Lukas Schneider	user4-demo@ntw.at	User
Beatriz Oliveira	user5-demo@ntw.at	User
Leonardo Barbacoa	barbacoa@gmx.at	User
Roger Batista	user7-demo@ntw.at	User
Amanda Moura	user8-demo@ntw.at	User
Sophie Gruber	user9-demo@ntw.at	User
Markus Steiner	user10-demo@ntw.at	User
Teste (XSS-Test)	user11-demo@ntw.at	User
Amanda Silva	user12-demo@ntw.at	User

7 Grenzen und Erweiterungen

7.1 Bekannte Grenzen der Version 1

- Die Anwendung läuft nur lokal. Hosting im Internet wurde nicht umgesetzt, auch wenn es dazu ein Ticket gibt (siehe 7.2).
- Der Datenbankzugang hat kein Passwort, und HTTPS läuft nur mit dem lokalen mkcert-Zertifikat. Das ist für Entwicklung und Präsentation gedacht, nicht für den echten Betrieb.
- CSS und JavaScript stehen in jeder PHP-Datei einzeln (siehe 3.2). Land und Stadt sind freie Texte, und die Oberflächentexte stehen in jeder Datei statt in zentralen Sprachdateien.
- Kunden können nicht deaktiviert und wieder aktiviert werden (ungenutzter Code wurde entfernt).
- Es gibt keine automatischen Tests. Getestet wurde manuell mit einem Testskript mit 132 Punkten.

7.2 Spätere Erweiterungen

- **Hosting:** Veröffentlichung im Internet mit eigener Domain, echtem Zertifikat (Let's Encrypt) und einem Datenbankbenutzer mit starkem Passwort.
- **CSS und JavaScript** in eigene Dateien auslagern (*style.css*, *app.js*); nur die Farbvariablen bleiben dynamisch.

- Zentrale Sprachdateien statt Texten in jeder PHP-Datei.
- Passwort vergessen (Zurücksetzen per E-Mail).
- Warnung bei schlechtem Kontrast in den Farbeinstellungen.
- Formale Sicherheitsprüfung der gesamten Anwendung.